AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Damian Nazwisko

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Założenia pierwszego etapu projektu 3

1.2. Środowisko programistyczne i narzędzia 3

2. Analiza problemu **4**

3. Projektowanie **5**

3.1. Zarządzanie wersjami w systemie Git 5

4. Implementacja **6**

4.1. Klasa węzła (`Node.h`) 6

4.2. Interfejs listy (`List.h`) 6

4.3. Definicja metod listy (`List.cpp`) 7

4.4. Testowanie działania (`main.cpp`) 8

5. Wnioski **10**

Literatura **11**

Spis rysunków **12**

Spis tabel **13**

Spis listingów **14**

1. Ogólne określenie wymagań

Celem projektu jest zaprojektowanie oraz implementacja dynamicznej struktury danych — listy dwukierunkowej — działającej na stercie w języku C++[1]. Dodatkowym założeniem pracy jest praktyczne wykorzystanie rozproszonego systemu kontroli wersji Git we współpracy z platformą GitHub[2] oraz zautomatyzowane generowanie dokumentacji technicznej przy użyciu narzędzia Doxygen [3].

1.1. Założenia pierwszego etapu projektu

Praca nad projektem została podzielona na etapy. W ramach pierwszego etapu zrealizowano fundamentalną architekturę programu obejmującą:

- podział programu na osobne moduły (pliki nagłówkowe `.h` oraz źródłowe `.cpp`),
- definicję klasy pojedynczego węzła listy (`Node`),
- definicję klasy zarządzającej listą dwukierunkową (`List`),
- alokację elementów na stercie z wykorzystaniem operatora `new`,
- dodawanie nowego elementu na początek listy (`addFront`),
- dodawanie nowego elementu na koniec listy (`addBack`),
- wyświetlanie całej zawartości listy w konsoli (`display`),
- inicjalizację repozytorium Git i rejestrowanie postępów pracy w kolejnych zatwierdzeniach (commitach).

1.2. Środowisko programistyczne i narzędzia

Do realizacji projektu wykorzystano zintegrowane środowisko programistyczne **Microsoft Visual Studio**, wyposażone w kompilator języka C++ oraz wbudowaną wtyczkę do obsługi systemu kontroli wersji Git i platformy GitHub. Dokumentację projektową przygotowano w systemie składu tekstu LaTeX.

[1] Opis zarządzania pamięcią dynamiczną znajduje się w literaturze [1].
[2] Dokumentacja systemu dostępna na stronie [2].

2. Analiza problemu

Lista dwukierunkowa (ang. *doubly linked list*) to dynamiczna struktura danych składająca się z sekwencji rekordów nazywanych węzłami (ang. *nodes*). W przeciwieństwie do tablic statycznych, elementy listy dwukierunkowej nie muszą zajmować ciągłego obszaru w pamięci operacyjnej komputera [4].

Każdy pojedynczy węzeł przechowuje trzy kluczowe składowe:

1. wartość danych (w realizowanym projekcie jest to liczba całkowita typu `int`),
2. wskaźnik `next` przechowujący adres kolejnego elementu na liście,
3. wskaźnik `prev` przechowujący adres poprzedniego elementu na liście.

Dostęp do całej struktury odbywa się za pomocą dwóch głównych wskaźników przechowywanych w klasie listy:

- `head` — wskazuje na pierwszy element listy (głowę),
- `tail` — wskazuje na ostatni element listy (ogon).

Jeżeli lista jest pusta, zarówno wskaźnik `head`, jak i `tail` przyjmują wartość pustą `nullptr`. Podobnie wskaźnik `prev` pierwszego elementu oraz wskaźnik `next` ostatniego elementu zawsze wskazują na `nullptr`, co wyznacza granice struktury.

Dzięki dwukierunkowym powiązaniom możliwe jest dodawanie nowych elementów zarówno na początku, jak i na końcu listy w stałym czasie obliczeniowym $\mathcal{O}(1)$, bez konieczności przesuwania pozostałych elementów w pamięci.

3. Projektowanie

Zgodnie z wymaganiami projektowymi, architektura aplikacji została zaprojektowana w sposób modułowy. Każda klasa znajduje się w osobnym pliku, co zwiększa czytelność kodu oraz ułatwia jego późniejszą rozbudowę w drugim etapie projektu. Podział plików przedstawiono w tabeli 3.1 (s. 5).

Tab. 3.1. Podział modułowy plików w pierwszym etapie projektu

Nazwa pliku	**Opis zawartości i przeznaczenie**
`Node.h`	Deklaracja i definicja klasy pojedynczego węzła listy
`List.h`	Plik nagłówkowy z deklaracją klasy listy i jej metod
`List.cpp`	Implementacja metod klasy `List` operujących na stercie
`main.cpp`	Funkcja główna testująca działanie zaimplementowanych metod

3.1. Zarządzanie wersjami w systemie Git

Praca w pierwszym etapie została podzielona na logiczne kroki rejestrowane w repozytorium Git za pomocą wtyczki w środowisku Visual Studio:

- **Commit 1:** *Dodanie klasy Node oraz podstawowego szkieletu klasy List* — utworzenie struktury projektu, konstruktora oraz destruktora listy.
- **Commit 2:** *Implementacja dodawania elementow na poczatek i koniec oraz wyswietlania listy* — rozbudowa klasy o metody `addFront`, `addBack` oraz `display` wraz z testami w pliku `main.cpp`.

4. Implementacja

W niniejszym rozdziale przedstawiono kod źródłowy zrealizowany w ramach pierwszego etapu projektu. Wszystkie węzły tworzone są dynamicznie na stercie za pomocą operatora `new`.

4.1. Klasa węzła (`Node.h`)

Kod 1 (s. 6) przedstawia implementację klasy `Node`, której konstruktor inicjalizuje wartość liczbową oraz ustawia wskaźniki sąsiedztwa na `nullptr`.

```
#ifndef NODE_H
#define NODE_H

class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};

#endif
```

Listing 1. Zawartość pliku nagłówkowego Node.h

4.2. Interfejs listy (`List.h`)

Kod 2 (s. 6) prezentuje plik nagłówkowy klasy `List` przechowującej prywatne wskaźniki `head` i `tail` oraz publiczne deklaracje metod.

```
#ifndef LIST_H
#define LIST_H

#include "Node.h"

class List {
private:
    Node* head;
```

```
9       Node* tail;
10
11  public:
12      List();
13      ~List();
14
15      void addFront(int val);
16      void addBack(int val);
17      void display();
18  };
19
20  #endif
```

Listing 2. Zawartość pliku nagłówkowego List.h

4.3. Definicja metod listy (`List.cpp`)

W pliku `List.cpp` (Kod 3, s. 7) zaimplementowano logikę przepinania wskaźników podczas dodawania elementów na krańcach listy oraz iteracyjnego wypisywania zawartości struktury.

```
1   #include <iostream>
2   #include "List.h"
3
4   using namespace std;
5
6   List::List() {
7       head = nullptr;
8       tail = nullptr;
9   }
10
11  List::~List() {
12      // Czyszczenie pamieci dodamy pozniej
13  }
14
15  void List::addFront(int val) {
16      Node* newNode = new Node(val);
17
18      if (head == nullptr) {
19          head = newNode;
20          tail = newNode;
21      } else {
22          newNode->next = head;
23          head->prev = newNode;
24          head = newNode;
```

```
25      }
26  }
27
28  void List::addBack(int val) {
29      Node* newNode = new Node(val);
30
31      if (tail == nullptr) {
32          head = newNode;
33          tail = newNode;
34      } else {
35          tail->next = newNode;
36          newNode->prev = tail;
37          tail = newNode;
38      }
39  }
40
41  void List::display() {
42      if (head == nullptr) {
43          cout << "Lista jest pusta." << endl;
44          return;
45      }
46
47      Node* current = head;
48      cout << "Zawartosc listy: ";
49      while (current != nullptr) {
50          cout << current->data << " ";
51          current = current->next;
52      }
53      cout << endl;
54  }
```

Listing 3. Zawartość pliku źródłowego List.cpp

4.4. Testowanie działania (`main.cpp`)

Działanie zaimplementowanych metod przetestowano w funkcji głównej programu przedstawionej w Kodzie 4 (s. 8).

```
1  #include <iostream>
2  #include "List.h"
3
4  using namespace std;
5
6  int main() {
7      List list;
```

```
8
9       cout << "--- Dodawanie elementow ---" << endl;
10      list.addFront(10);
11      list.addFront(5);
12      list.addBack(20);
13      list.addBack(30);
14      list.display();
15
16      return 0;
17  }
```

Listing 4. Zawartość pliku głównego main.cpp

5. Wnioski

W ramach pierwszego etapu projektu pomyślnie zaprojektowano oraz zaimplementowano podstawową strukturę listy dwukierunkowej działającej na stercie w języku C++. Podział kodu na osobne pliki nagłówkowe oraz źródłowe zapewnił przejrzystość architektury programu.

Testy przeprowadzone w funkcji `main` potwierdziły poprawność działania metod `addFront`, `addBack` oraz `display`. Po dodaniu wartości 10 i 5 na początek listy oraz wartości 20 i 30 na jej koniec, program prawidłowo wyświetlił w konsoli sekwencję: `5 10 20 30`.

Wykorzystanie wtyczki Git w środowisku Visual Studio umożliwiło bieżące śledzenie zmian w kodzie źródłowym oraz bezpieczne przesyłanie kolejnych wersji projektu (commitów) do zdalnego repozytorium na platformie GitHub. Przygotowana baza stanowi solidny fundament pod realizację drugiego etapu projektu, obejmującego m.in. usuwanie elementów, obsługę indeksów oraz implementację wzorców projektowych *Factory* i *Iterator*.

Bibliografia

[1] Jerzy Grębosz. *Opus magnum C++11. Programowanie w języku C++*. Gliwice: Helion, 2017.

[2] *Oficjalna dokumentacja systemu kontroli wersji Git*. URL: https://git-scm.com/doc (dostęp 20.10.2026).

[3] *Strona internetowa generatora dokumentacji Doxygen*. URL: https://www.doxygen.nl (dostęp 20.10.2026).

[4] *Dokumentacja techniczna języka C++*. URL: https://en.cppreference.com (dostęp 20.10.2026).

Spis rysunków

Spis tabel

3.1. Struktura plików projektu 5

Spis listingów

1. Zawartość pliku nagłówkowego Node.h 6
2. Zawartość pliku nagłówkowego List.h 6
3. Zawartość pliku źródłowego List.cpp 7
4. Zawartość pliku głównego main.cpp 8